

부하테스트 결과 보고서

트래픽 폭증 상황에서의 대기열(Rate Limiting) 적용 효과 검증

문서번호	LT-2026-0706-01	작성일	2026-07-06
대상 시스템	티켓 예매 서비스 — queue-gate(172.16.60.146)	테스트 도구	k6 (macOS, Homebrew 설치)
테스트 위치	맥북 → queue-gate (외부 네트워크 경유)	버전	1.0

1. 개요

티켓 예매 오픈 순간에는 조회·예약 요청이 짧은 시간에 집중되어 서버가 과부하 상태에 빠질 수 있다. 본 테스트는 queue-gate에 적용한 Rate Limiting(대기열)이 실제로 이러한 트래픽 폭증 상황에서 서비스 가용성을 유지시키는지 정량적으로 검증하기 위해 수행했다. 동일한 부하 패턴을 대기열 비활성화(OFF) 상태와 활성화(ON) 상태에서 각각 1회씩 실행하여 결과를 비교했다.

2. 테스트 환경

구성 요소	내용
queue-gate	Nginx 1.28.3, limit_req_zone(rate=10r/s, burst=20, nodelay), limit_conn_zone(5)
web-lb	Nginx, upstream least_conn (was-01, was-02)
was-01 / was-02	Node.js 20 / 22, Express, 좌석 예약 API
부하 발생 위치	맥북 (VMware Fusion 호스트, 172.16.60.1) — 대상 서버와 물리적으로 분리된 위치에서 실행

3. 테스트 시나리오

가상 사용자(VU)를 단계적으로 증가시켜 순간 폭증 상황을 재현했다. 대상 엔드포인트는 /api/health이며, 동일 스크립트를 대기열 OFF/ON 두 조건에서 각각 실행했다.

구간	지속 시간	목표 VU
1단계 (예열)	10s	50
2단계 (급증)	5s	300
3단계 (유지)	20s	300
4단계 (정리)	10s	0

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  stages: [
    { duration: '10s', target: 50 },
    { duration: '5s', target: 300 },
    { duration: '20s', target: 300 },
    { duration: '10s', target: 0 },
  ],
};

export default function () {
  const res = http.get('http://172.16.60.146/api/health');
  check(res, {
    '정상 또는 제한 응답': (r) => [200, 429, 503].includes(r.status),
  });
}
```

체크 조건은 200(정상), 429(Rate Limit 초과), 503(서버 과부하) 세 응답을 모두 "정상 범위"로 간주했다. 이는 대기열이 초과 요청을 명시적으로 거절(429)하는 것이 설계상 의도된 동작이기 때문이다.

4. 실행 절차

```
sudo cp /etc/nginx/conf.d/queue.conf /etc/nginx/conf.d/queue.conf.bak
sudo sed -i 's/limit_req zone=global/#limit_req zone=global/' /etc/nginx/conf.d/queue.conf
sudo nginx -t
sudo systemctl restart nginx
```

```
k6 run --out json=no-queue-result.json ~/traffic-spike.js
```

4.2 대기열 복원 및 ON 실행 (2차 실행)

```
sudo cp /etc/nginx/conf.d/queue.conf.bak /etc/nginx/conf.d/queue.conf
sudo nginx -t
sudo systemctl daemon-reload
sudo systemctl restart nginx

k6 run --out json=with-queue-result.json ~/traffic-spike.js
```

5. 실행 결과

5.1 대기열 OFF

```
checks_total.....: 91801   1469.664337/s
checks_succeeded....: 98.72%  90631 out of 91801
checks_failed.....: 1.27%   1170 out of 91801

http_req_duration...: avg=5.64ms min=0s med=2.87ms max=1m0s p(90)=3.44ms p(95)=3.94ms
http_req_failed.....: 96.06%  88192 out of 91801
http_reqs.....: 91801   1469.664337/s

running (1m02.5s), 000/300 VUs, 91801 complete and 0 interrupted iterations
```

5.2 대기열 ON

```
checks_total.....: 864696  19212.708576/s
checks_succeeded....: 100.00% 864696 out of 864696
checks_failed.....: 0.00%    0 out of 864696

http_req_duration...: avg=9.93ms min=44us med=12.4ms max=108.8ms p(90)=17.49ms p(95)=19.7ms
http_req_failed.....: 99.94%  864225 out of 864696
http_reqs.....: 864696  19212.708576/s

running (0m45.0s), 000/300 VUs, 864696 complete and 0 interrupted iterations
```

5.3 비교 요약

지표	대기열 OFF	대기열 ON	비고
checks_failed	1.27%	0.00%	완전 무응답(타임아웃) 요청 소멸
http_req_duration max	60,000ms	108.8ms	약 99.8% 감소
http_req_duration p95	3.94ms	19.7ms	정상 처리 구간의 p95는 오히려 증가(5.4절 참고)
처리 요청 수	91,801건	864,696건	약 9.4배
초당 처리량	1,469.7 req/s	19,212.7 req/s	-

6. 결과 분석

k6의 `http_req_failed` 지표는 HTTP 200이 아닌 모든 응답을 "실패"로 집계하므로, 이 지표만으로 대기열의 효과를 단순 비교하는 것은 오독의 소지가 있다. 대기열 ON 조건에서 `http_req_failed`가 오히려 99.94%로 더 높게 나타난 것은, `limit_req`의 `nodelay` 옵션에 의해 초과 요청이 대기 없이 즉시 429로 응답되어 처리량 자체가 급증했기 때문이다. 실제 서비스 품질을 판단하는 데 유효한 지표는 다음 두 가지다.

- `checks_failed` — "200 또는 429/503 중 하나라도 응답을 받았는가"를 기준으로 하며, 대기열 ON에서 0%를 기록해 모든 요청이 어떤 형태로든 응답을 받았음을 의미한다.
- `http_req_duration max` — 대기열 OFF에서는 서버가 처리 불가능한 요청을 큐에 쌓아두다 60초 타임아웃에 도달했으나, ON에서는 초과분을 즉시 거절하여 최대 응답시간이 108.8ms로 유지되었다.

결론적으로 대기열은 "요청을 느리게 처리"하는 방식이 아니라 "감당 가능한 양만 처리하고 초과분은 즉시 거절"하는 방식으로 서버 가용성을 지킨다. 이는 도입 시 설계 의도와 일치하는 결과다.

7. 결론 및 후속 조치

1. 대기열(Rate Limiting) 적용은 300 VU 규모의 순간 폭증 상황에서 서버 응답 지연을 60,000ms에서 108.8ms 수준으로 낮추는 효과가 확인되었다. 현재 설정(초당 10건, burst 20)을 유지한다.
2. 429 응답에 대한 클라이언트 측 안내(예: "잠시 후 다시 시도해 주세요" 화면)가 프론트엔드에 아직 반영되지 않았다. 다음 스프린트에서 처리할 항목으로 남긴다.

3. was-02 노드를 추가한 상태에서의 재측정, 그리고 VU를 2,000 이상으로 늘린 재현 테스트는 VM 리소스 제약으로 보류했다. 향후 서버 사양 확충 후 재검증이 필요하다.

작성: 최시은

검토: -

배포: 내부 참고용